CareerTrack — Minor Project 5

CSCE 547 · Windows Programming | Service Layer Project | Graduate Submission

Instructor: Professor Robert Bailey

Student: Divyashanu Swain

Date: October 31, 2026

GitHub repository URL: https://github.com/dswain07/CareerTrack-ServiceLayer

Documentation for the ASP.NET Core Web API, business and data layers, and browser client.

1. Project overview

CareerTrack is a local job application management system. It organizes employers, job applications, and interviews in a SQLite database and exposes the records through an ASP.NET Core Web API. A simple HTML/CSS/JavaScript client retrieves records and demonstrates company creation, editing, and deletion.

2. Architecture and implementation

The solution separates the API, business logic, and data access into CareerTrack.Api, CareerTrack.Business, and CareerTrack.Data. The API controllers accept HTTP requests and map database entities to client-facing service models. Business services enforce validation rules before delegating persistence to repositories. Repositories use Entity Framework Core and CareerTrackDbContext to access SQLite.

Layer / project	Responsibility
CareerTrack.Api	REST controllers, service models, and static HTML client
CareerTrack.Business	CompanyService, JobApplicationService, InterviewService; validation rules
CareerTrack.Data	Entity models, DbContext, repositories, EF Core/SQLite
CareerTrack.Client	Separate C# console HTTP client used for GET verification

The reused database contains Companies, JobApplications, Interviews, ApplicationStatuses, and Contacts. Full CRUD is implemented in the data, business, and API layers for Companies, JobApplications, and Interviews. The repository includes DDL.sql and DML.sql for database recreation.

3. API and client demonstration

The browser client retrieves all three record types and individual records. Its Company section supports adding, viewing, editing, and deleting a record. API tests also verified write operations for job applications and interviews. The test client uses the local API at http://localhost:5043 during development.

4. Graduate requirement: five business rules

Rule	Enforcement and verification
1. Unique company names	CompanyService rejects duplicate names after trimming and case-insensitive comparison; HTTP 409 verified.
2. Protect companies with applications	CompanyService blocks deletion if associated job applications exist; HTTP 409 verified.
3. Nonnegative salary	JobApplicationService rejects a negative salary on create/update; HTTP 400 verified.
4. Valid application relationships	JobApplicationService requires existing CompanyId and ApplicationStatusId; both HTTP 400 tests verified.
5. Valid interview relationship	InterviewService requires an existing JobApplicationId on create/update; HTTP 400 verified.

5. Graduate requirement: service model design choices

The service model classes are CompanyModel, JobApplicationModel, and InterviewModel. They represent the data needed by the client rather than exposing EF Core entity graphs. Each includes an Id so the client can request individual records and target updates or deletions.

CompanyModel includes Name, Industry, and Location because these are displayed in the company list, details, and edit form. The database entity's Website and related navigation data are excluded because the current client does not use them.

JobApplicationModel includes JobTitle, CompanyId, ApplicationStatusId, DateApplied, Location, JobUrl, Salary, and Notes. The IDs preserve links to existing lookup records and support business validation; the remaining fields support list and details views. Company and status navigation objects are excluded to keep JSON responses concise and avoid nested entity relationships.

InterviewModel includes InterviewType, InterviewDate, Location, Outcome, Notes, and JobApplicationId so the client can show interview details and associate an interview with its application. The JobApplication navigation object is excluded because the ID is sufficient for the current client and the API can retrieve the related record separately.

6. Required reflection questions

6.1 Describe a challenge you overcame in this project

A major challenge was connecting the separate C# client to the locally hosted Web API. Initially, the client received a connection-refused error because the API was not reachable at the configured HTTPS address. After starting the API independently, the client reached the server but encountered an SSL certificate validation error. Although the development certificate existed, the trust check still reported it as untrusted. To continue testing locally, I used the API's HTTP development endpoint at localhost:5043 and temporarily disabled HTTPS redirection. This allowed the client to communicate successfully with the service. I also fixed a stylesheet filename mismatch that caused the browser client to appear unstyled. These issues taught me to distinguish network, certificate, and static-file problems from application logic errors.

6.2 Describe a technique you learned in this project

I learned how to structure a Web API using dependency injection and separate data, business, and service layers. The repositories receive CareerTrackDbContext through constructor injection and perform

asynchronous EF Core operations. Business services receive repositories, validate incoming data, and delegate persistence. Controllers receive business services, map entities into service models, and return meaningful HTTP status codes such as 200, 201, 204, 400, 404, and 409. I also practiced testing REST endpoints with HTTP request files and verifying changes with follow-up GET requests.

6.3 What would you refine before selling, shipping, or deploying?

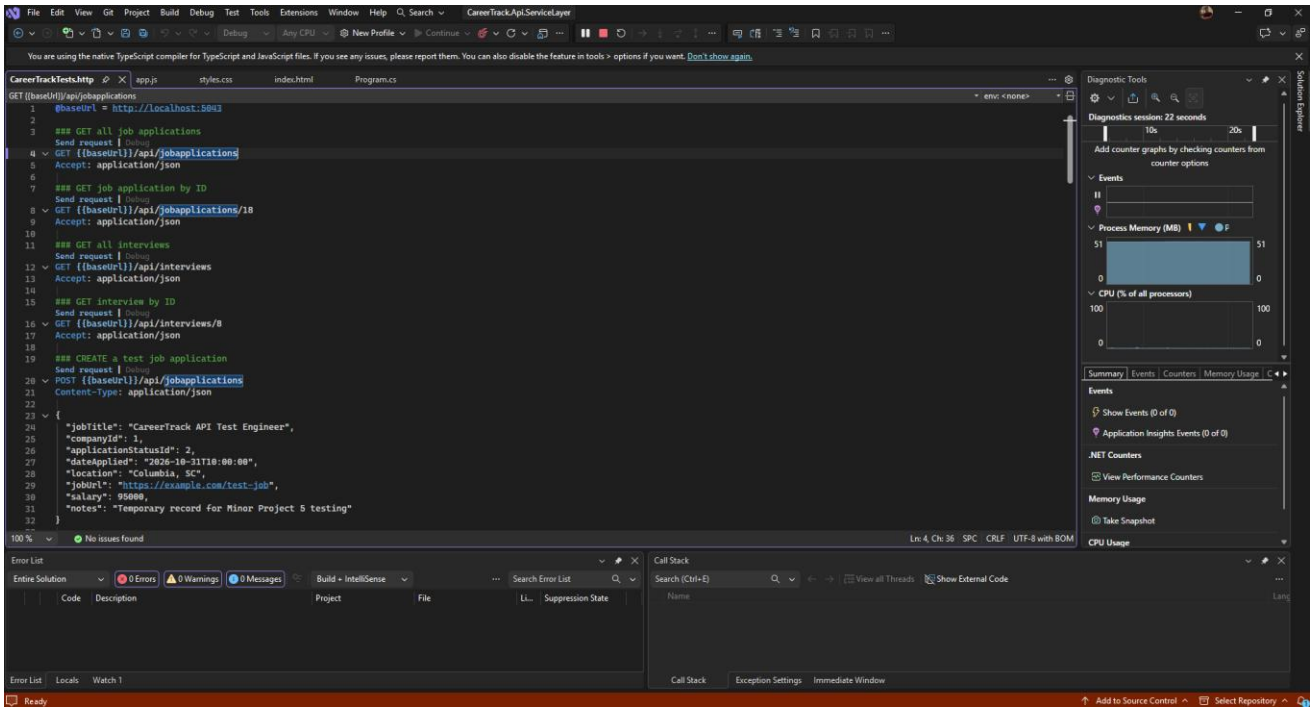
Before deployment, I would enable trusted HTTPS, add authentication and authorization, move the database to a managed and backed-up environment, improve logging and exception handling, and add automated unit and integration tests. I would enforce company-name uniqueness in the database to handle concurrent requests, refine delete behavior for related records, and add pagination and server-side filtering. For daily use, I would complete create/edit/delete forms for job applications and interviews, add status labels, follow-up reminders, document attachments, and more actionable dashboard analytics. I would also review accessibility, input validation, configuration management, and deployment security.

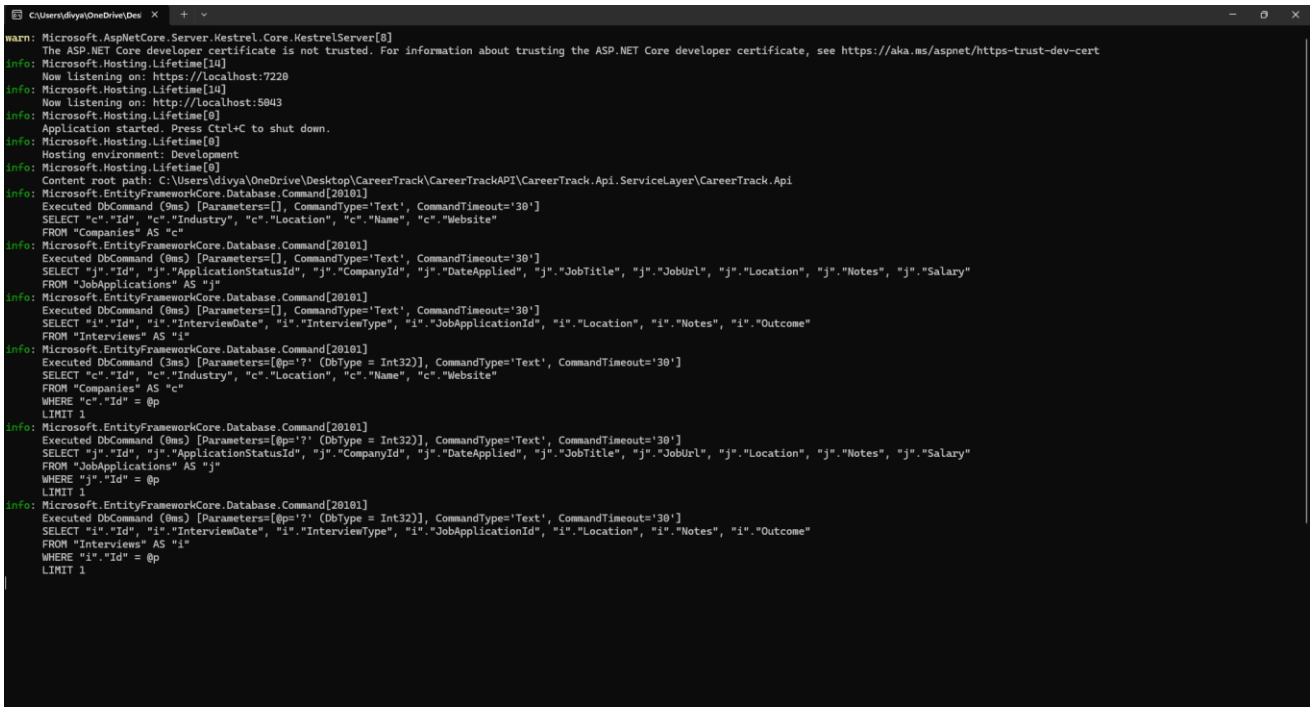
7. Verification summary

Observed testing: GET list and GET by ID succeeded for all three entities; browser Company CRUD succeeded; JobApplication and Interview POST, PUT, and DELETE succeeded using temporary test records; the deleted records returned 404 afterward. All six negative tests passed, covering the five business rules. Screenshots below should be inserted from the actual running project as evidence.

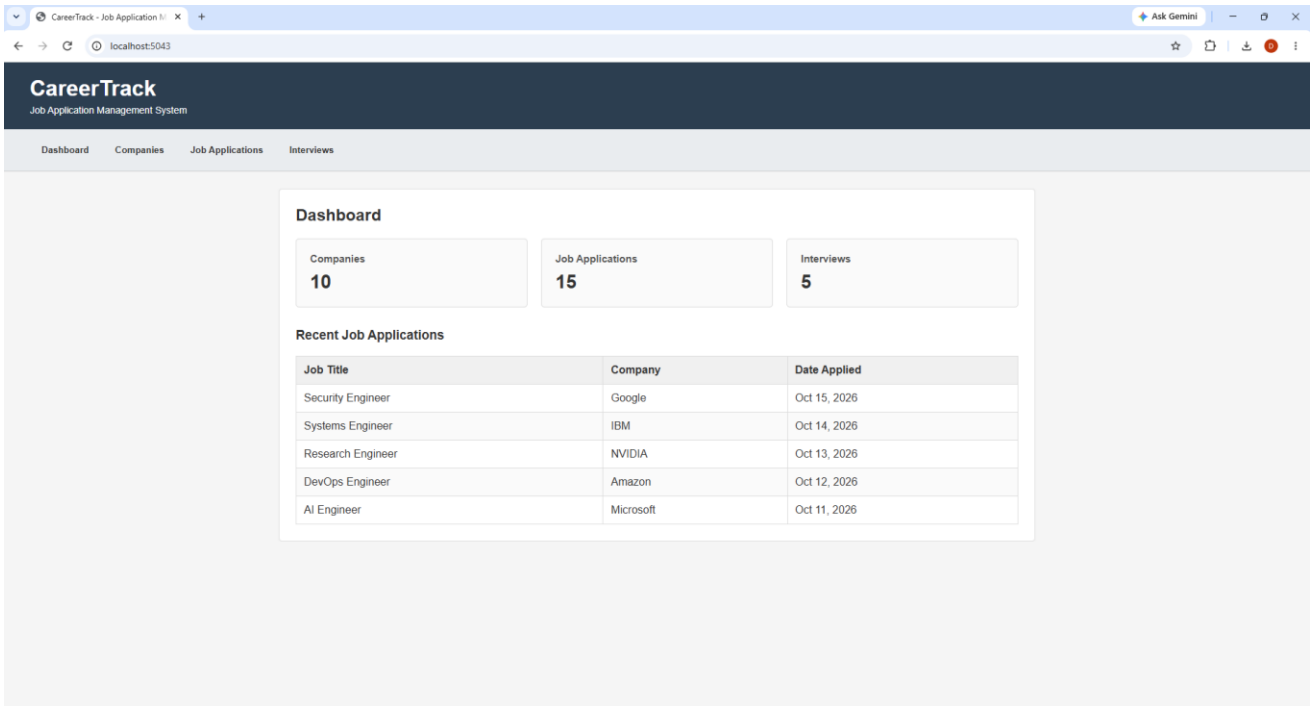
8. Screenshot evidence

8.1 Service running





8.2 Client dashboard and record counts



8.3 Companies — GetAll and GetById



Figure 1. GET All on Company



Figure 2. GET by ID on Company

8.4 Job Applications — GetAll and GetById



Figure 3. GET All on Job Application



Figure 4. GET by ID on Job Application

8.5 Interviews — GetAll and GetById



Figure 5. GET All on Interview



Figure 6. GET by ID on Interview

8.6 Company Create, Update, Delete

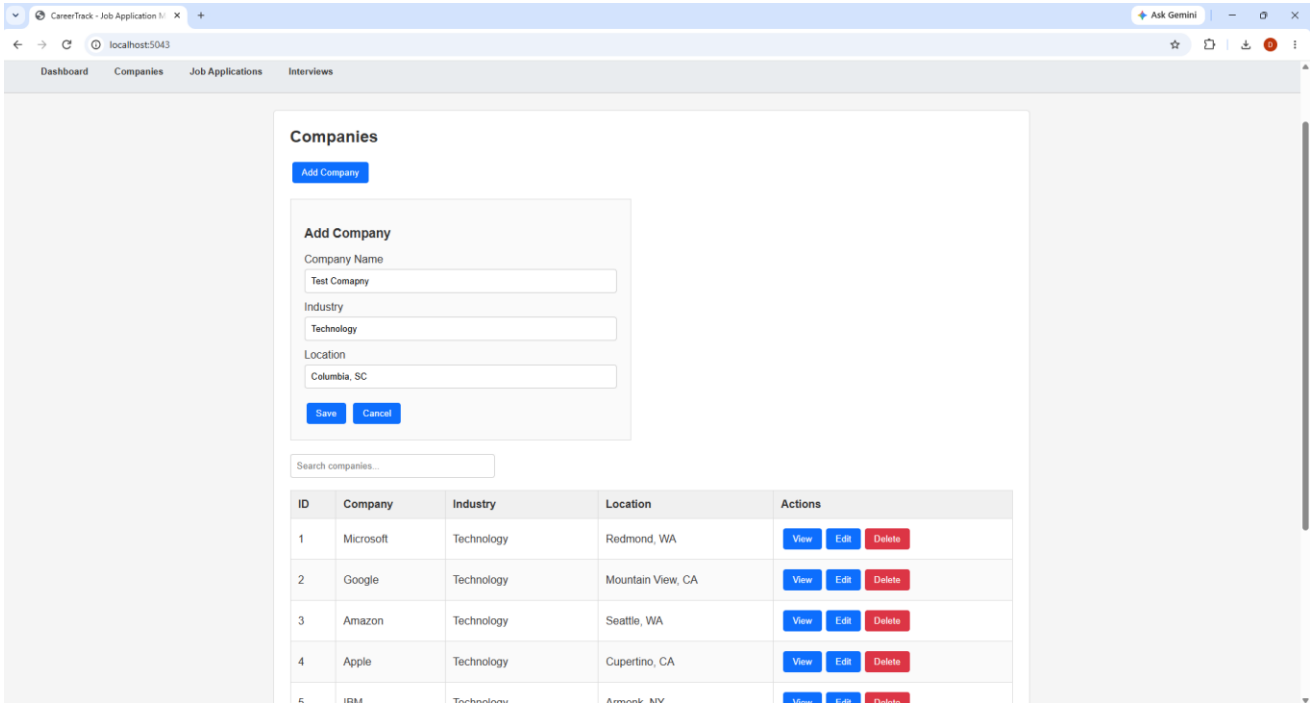


Figure 7. Create (POST) on Company

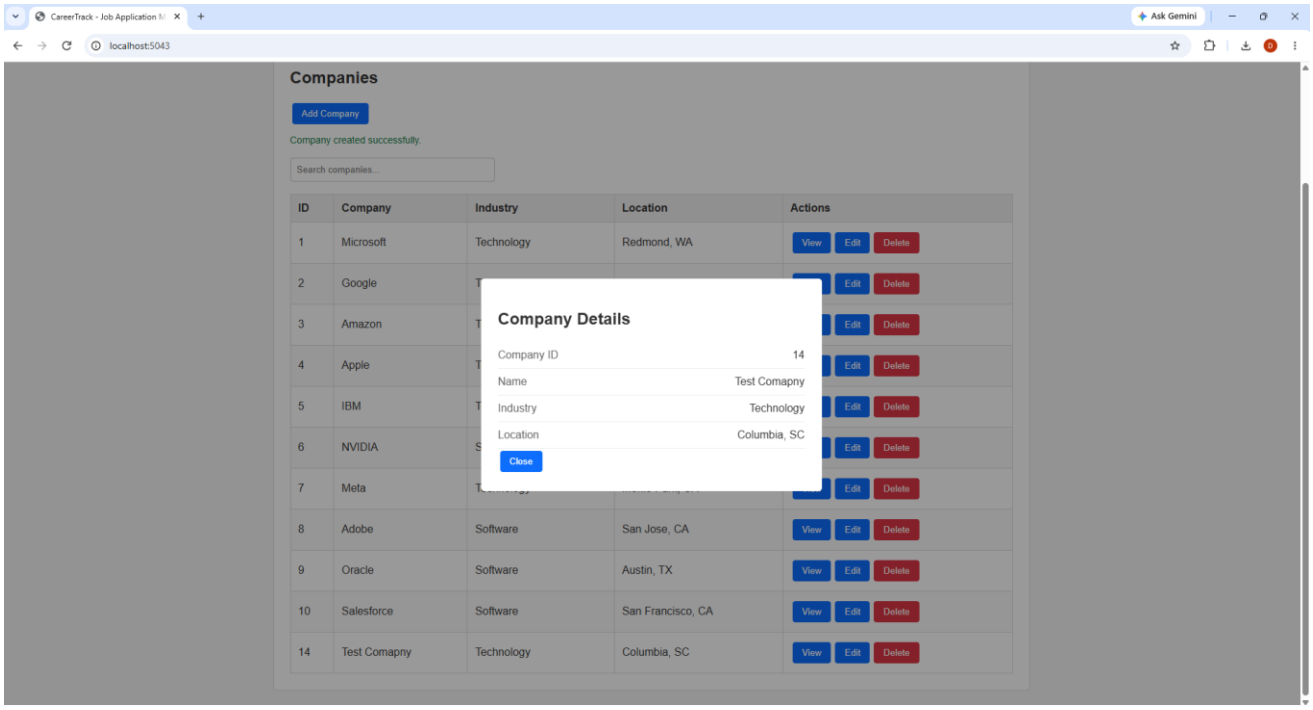


Figure 8. Read on Company

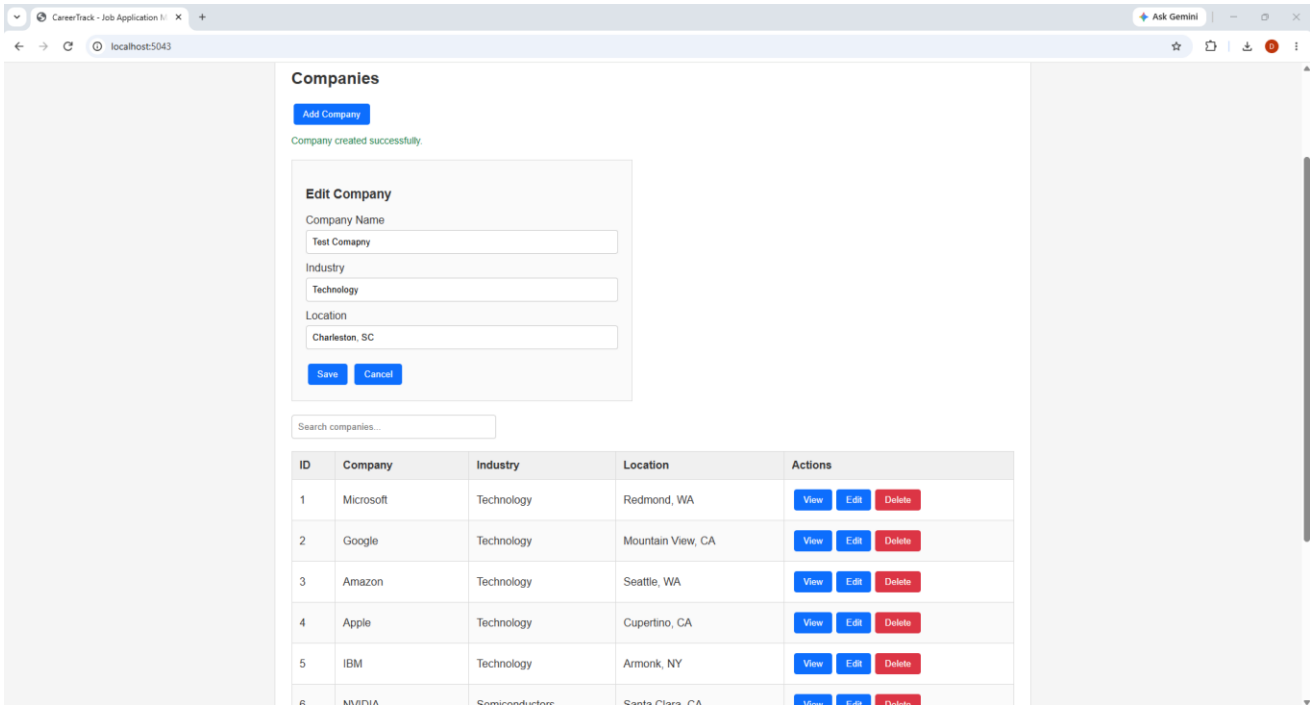


Figure 9. Update on Company

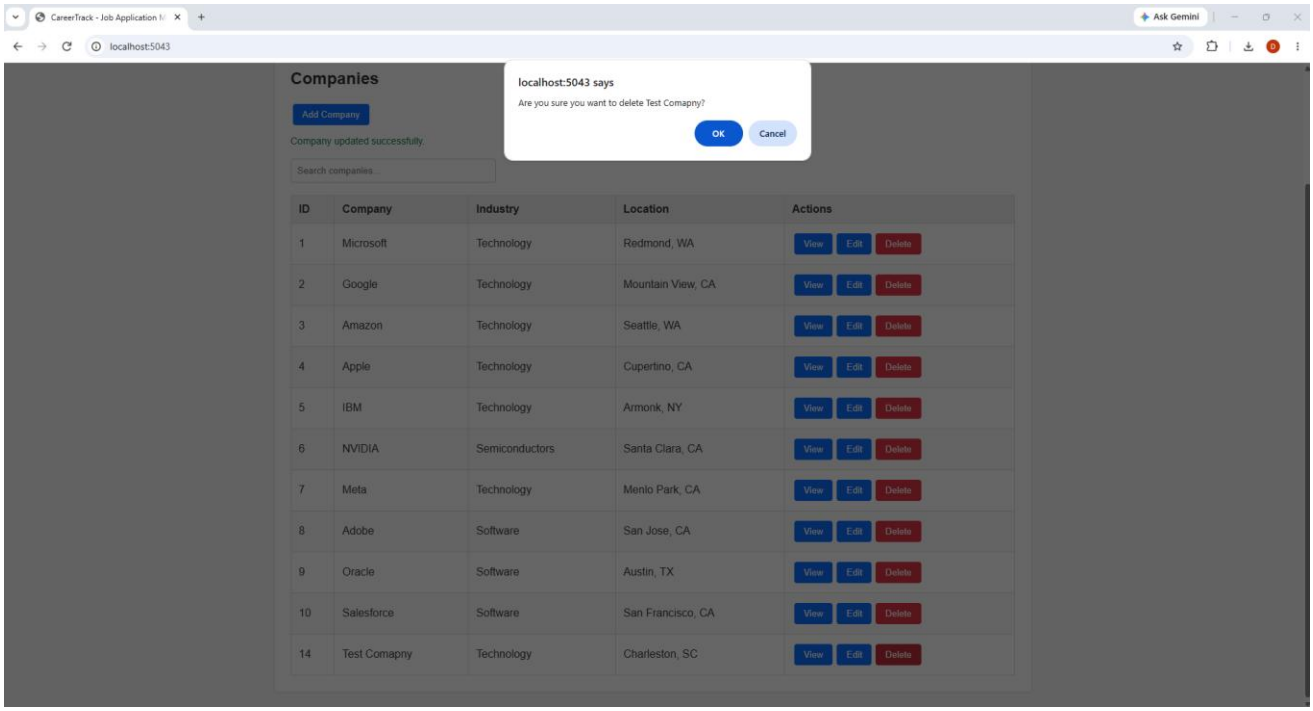


Figure 10. Delete on Company

8.7 Job Application CRUD responses

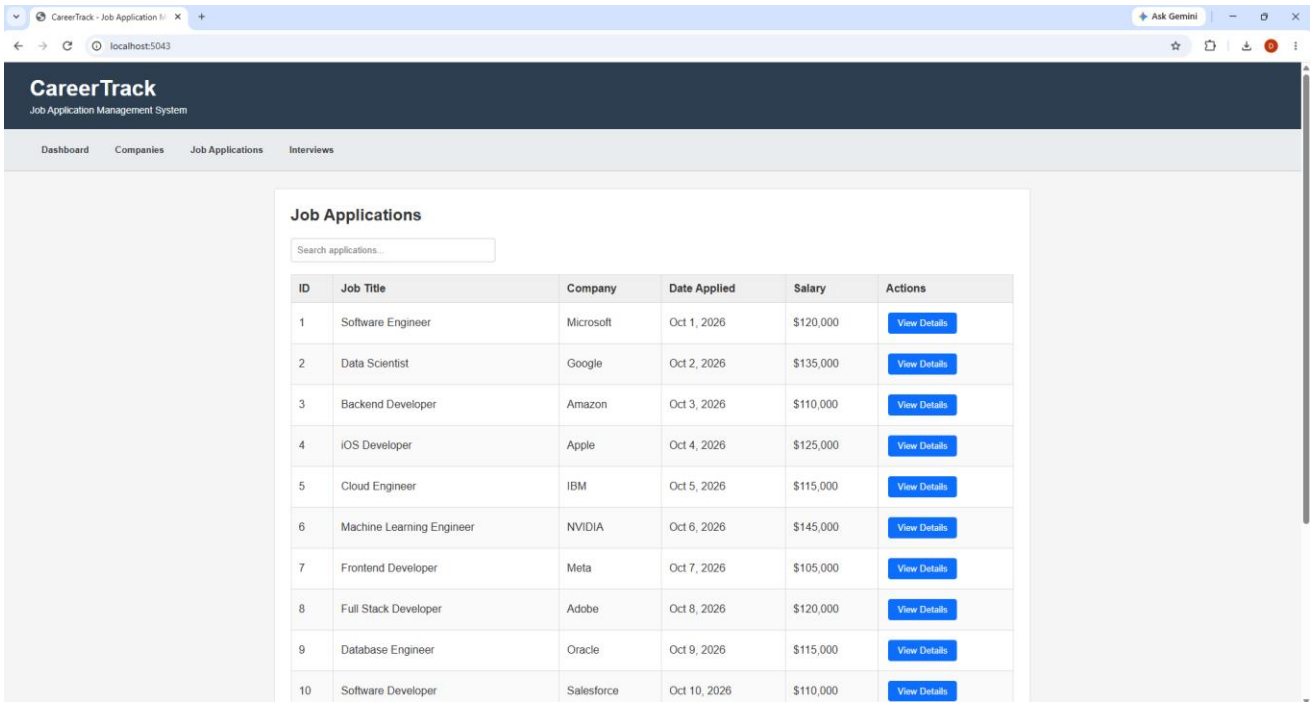
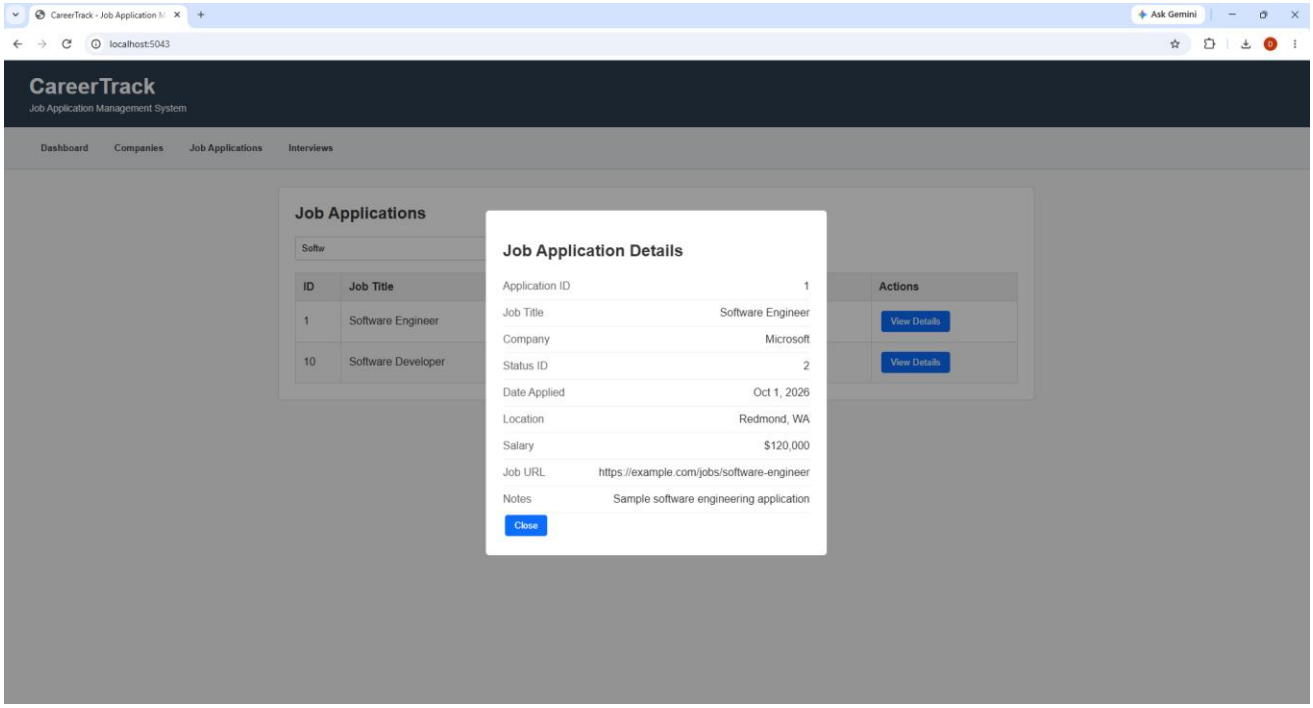


Figure 11. Get on Job Application



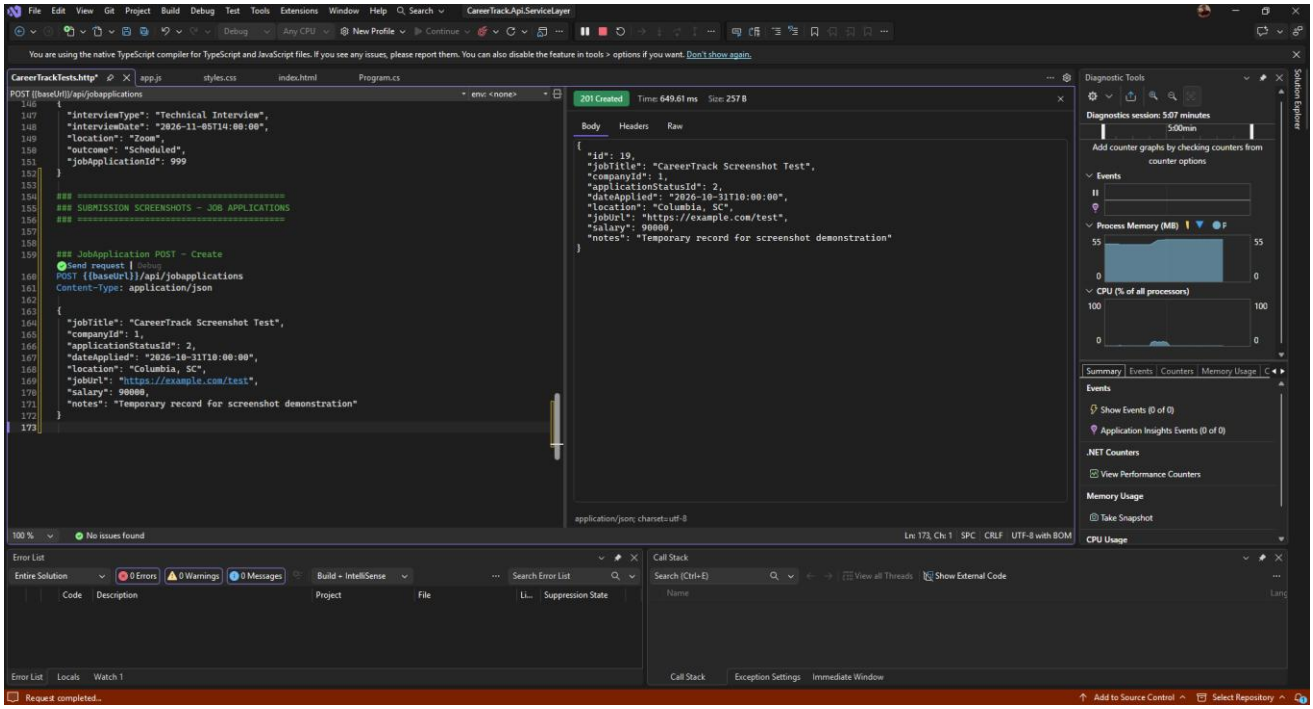


Figure 12. POST on Job Application

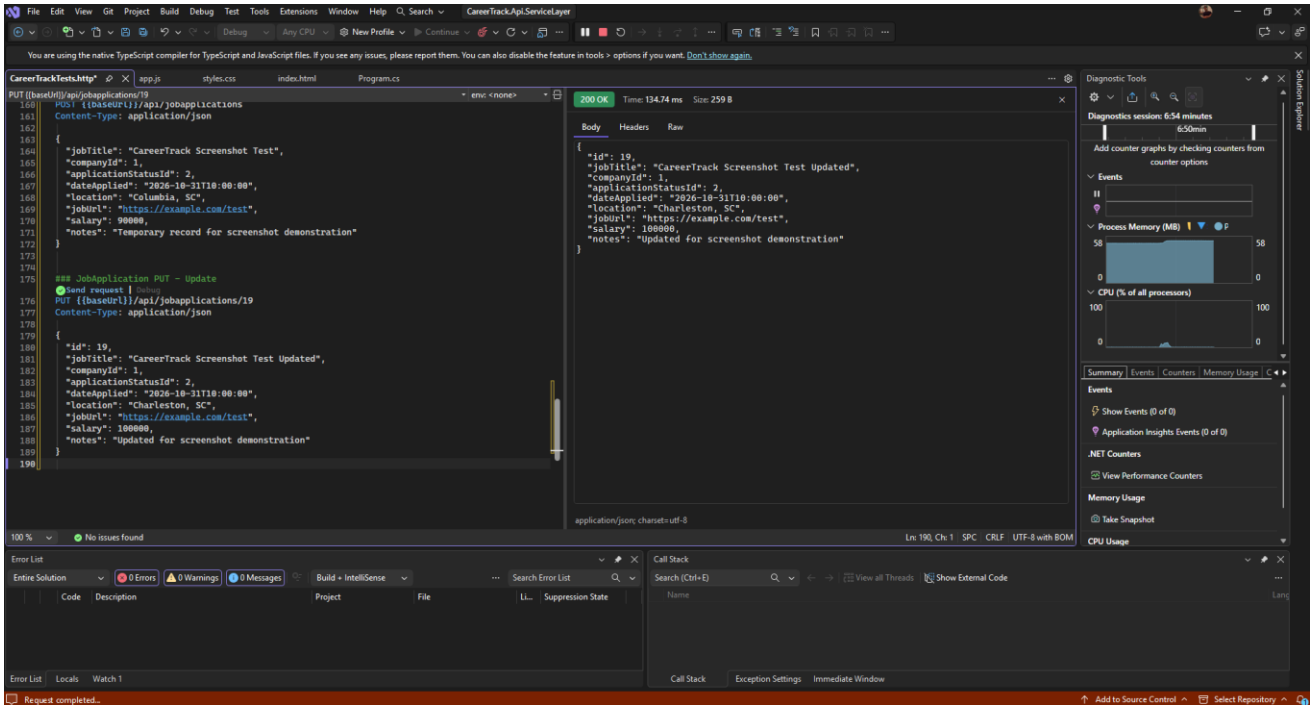


Figure 13. PUT on Job Application

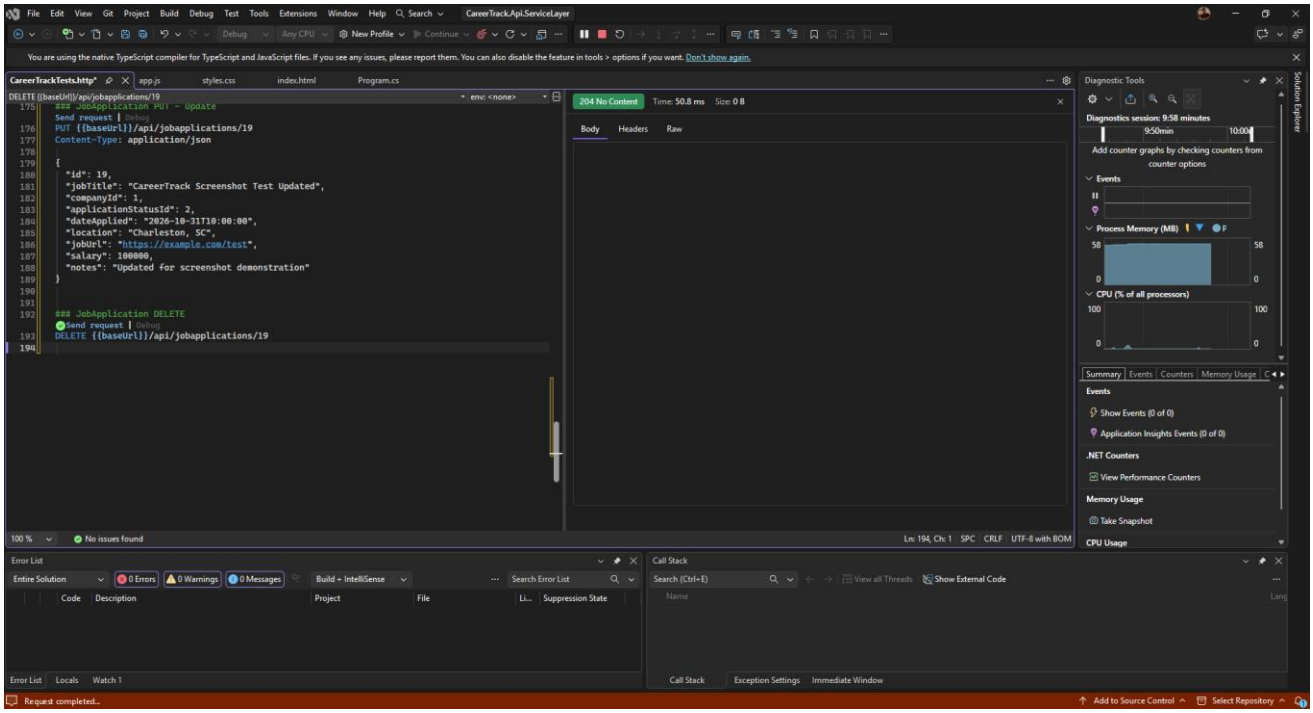
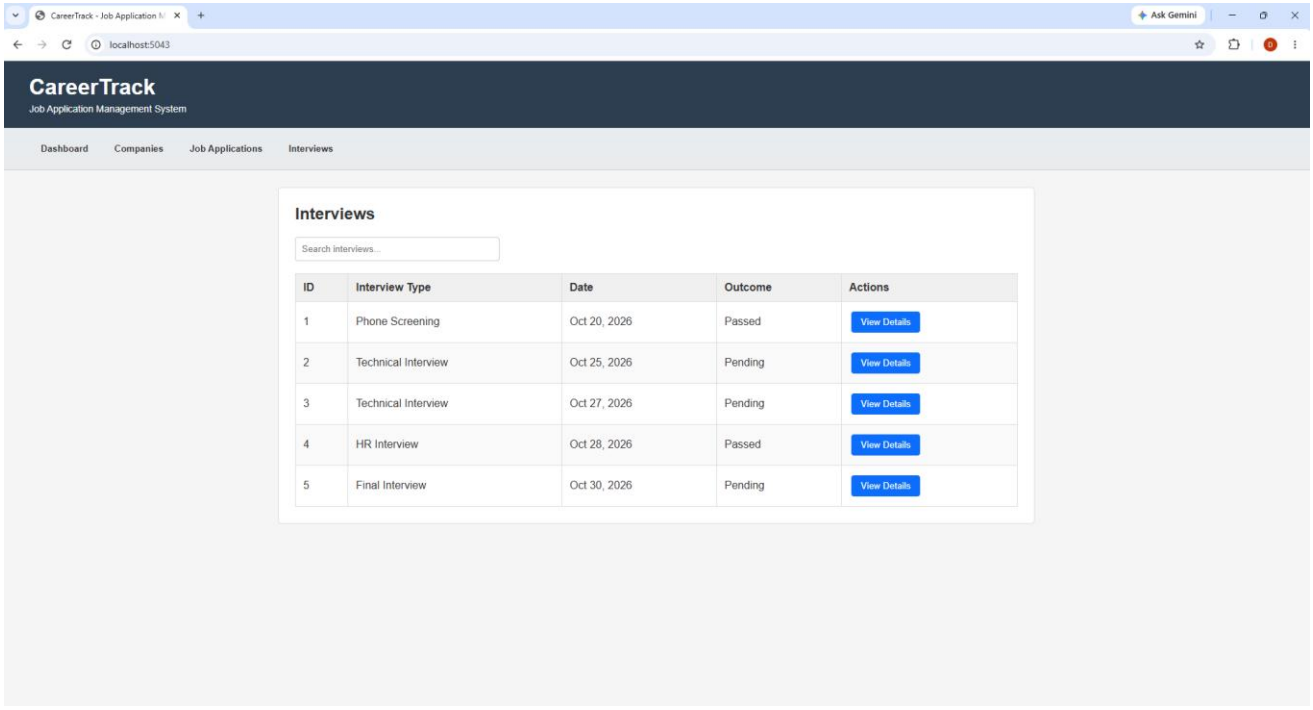


Figure 14. Delete on Job Application

8.8 Interview CRUD responses



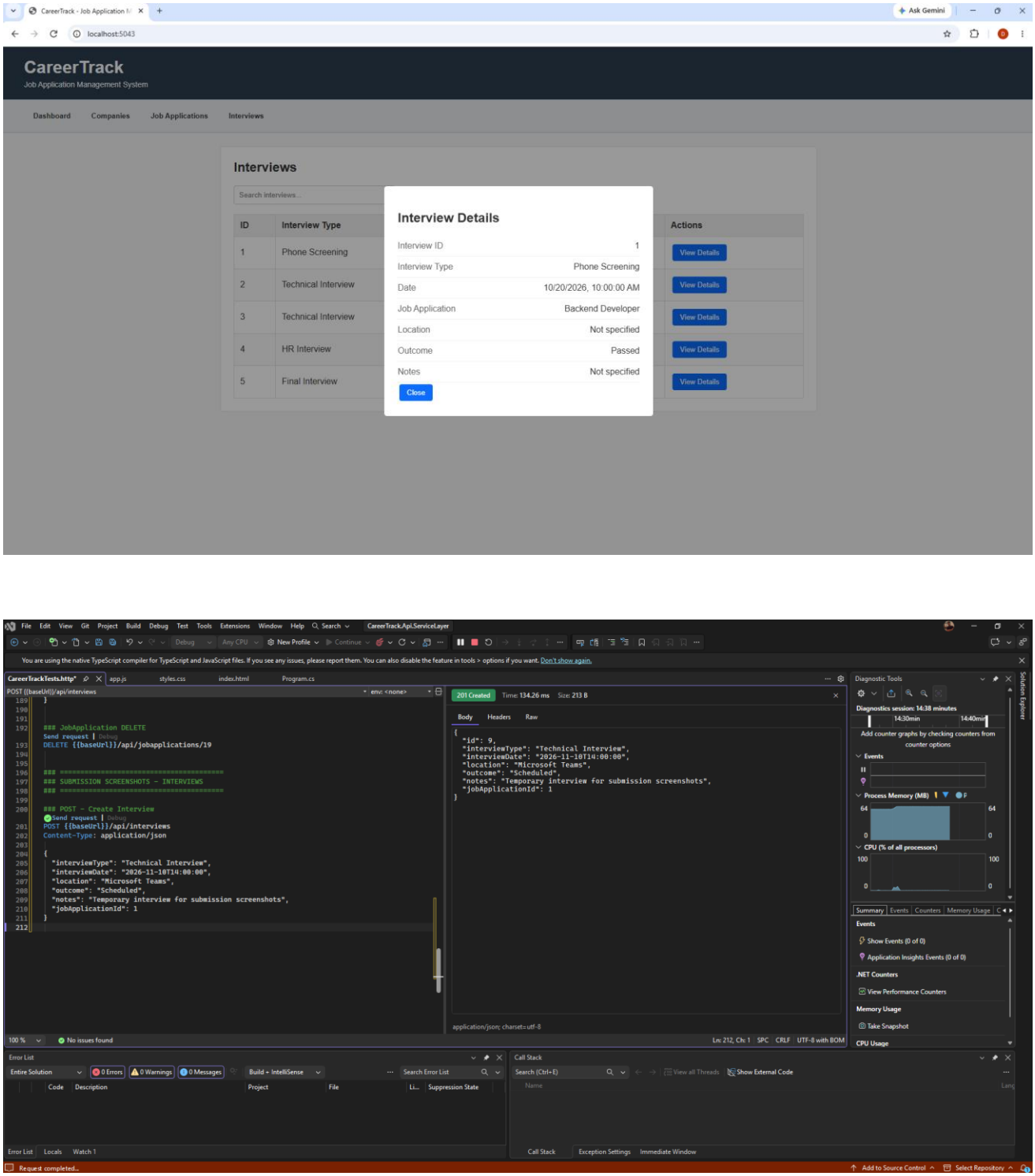


Figure 15. POST on Interview

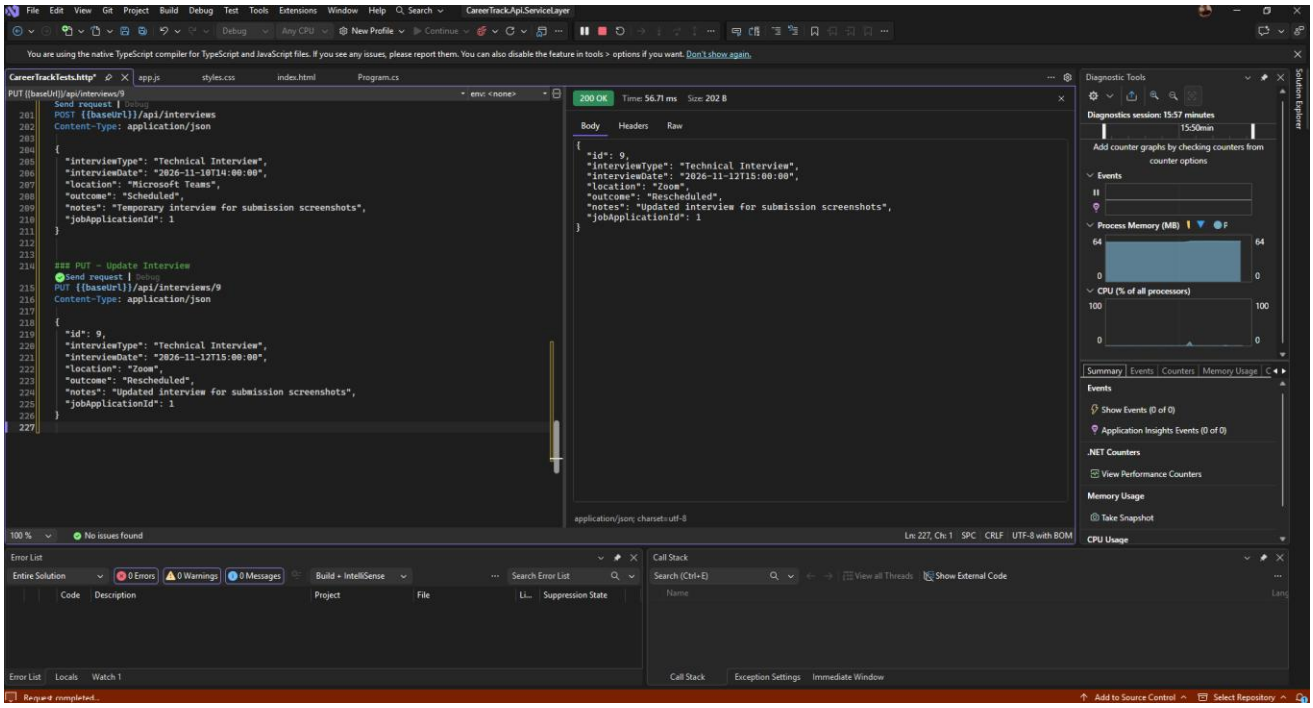


Figure 16. PUT on Interview

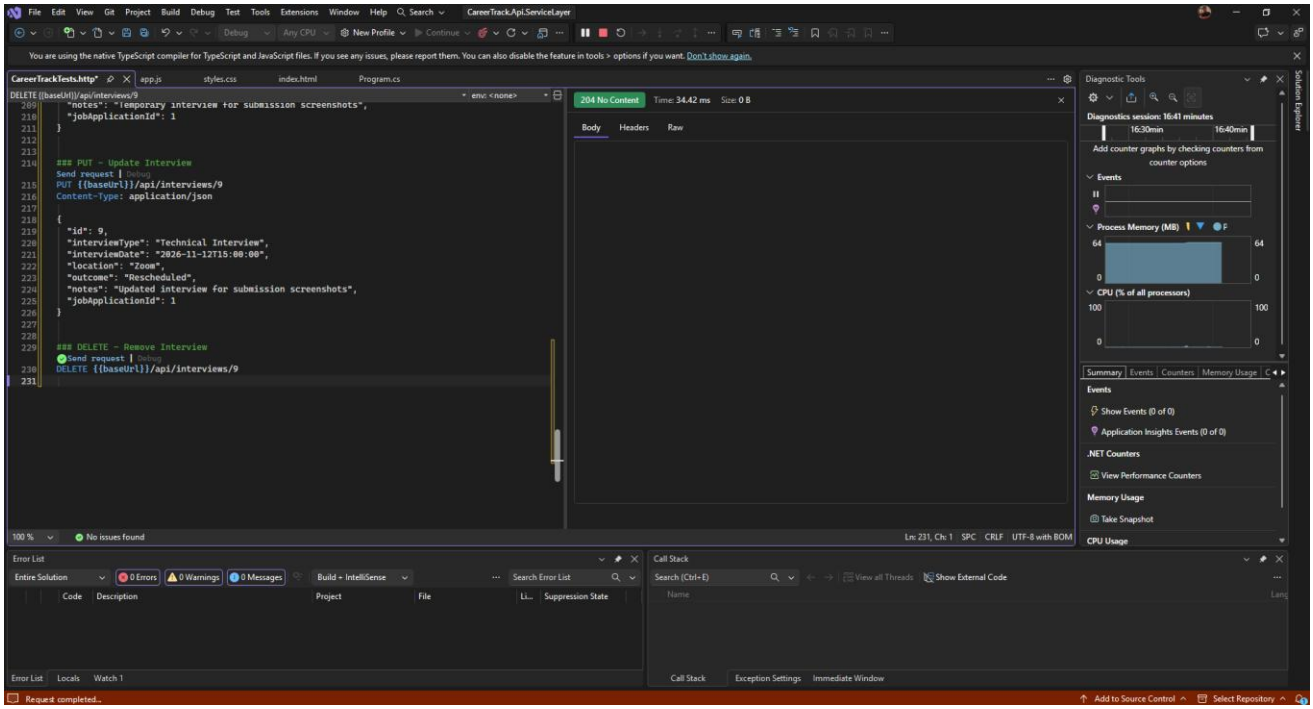


Figure 17. Delete on Interview

8.9 Graduate business-rule failures

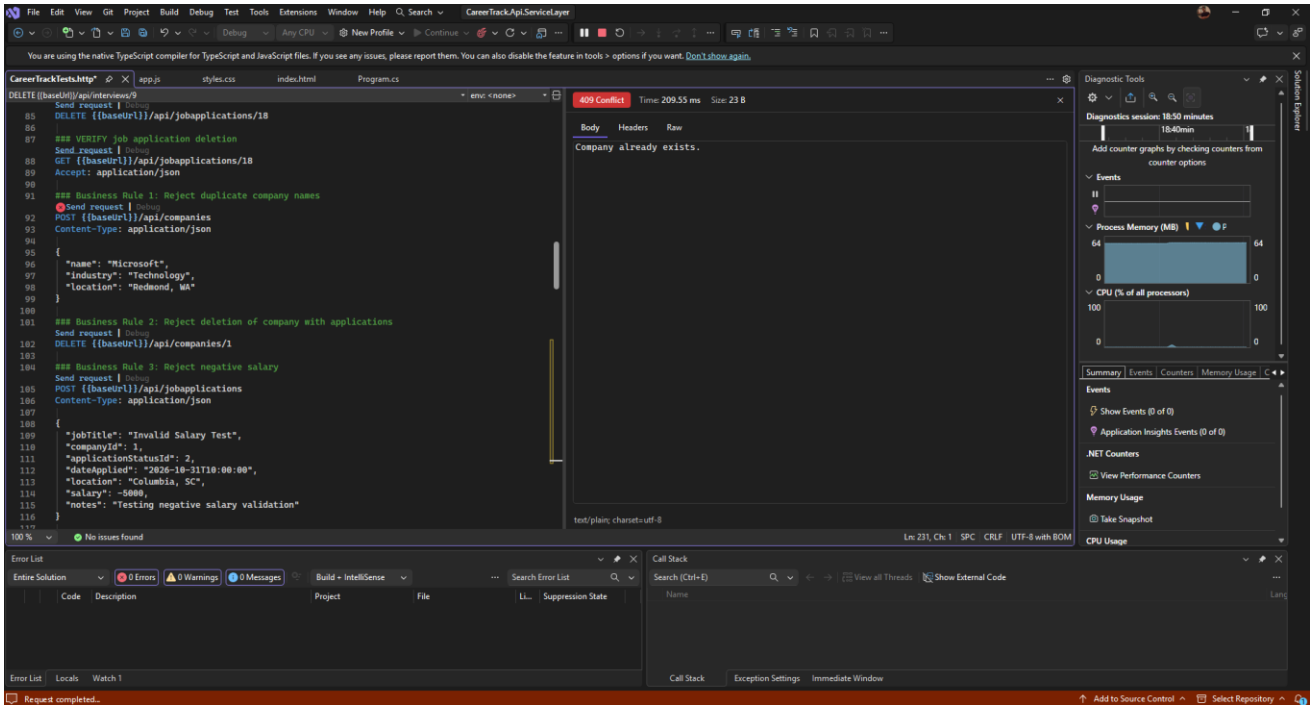


Figure 18. Rule 1 - Reject Duplicate Company Name

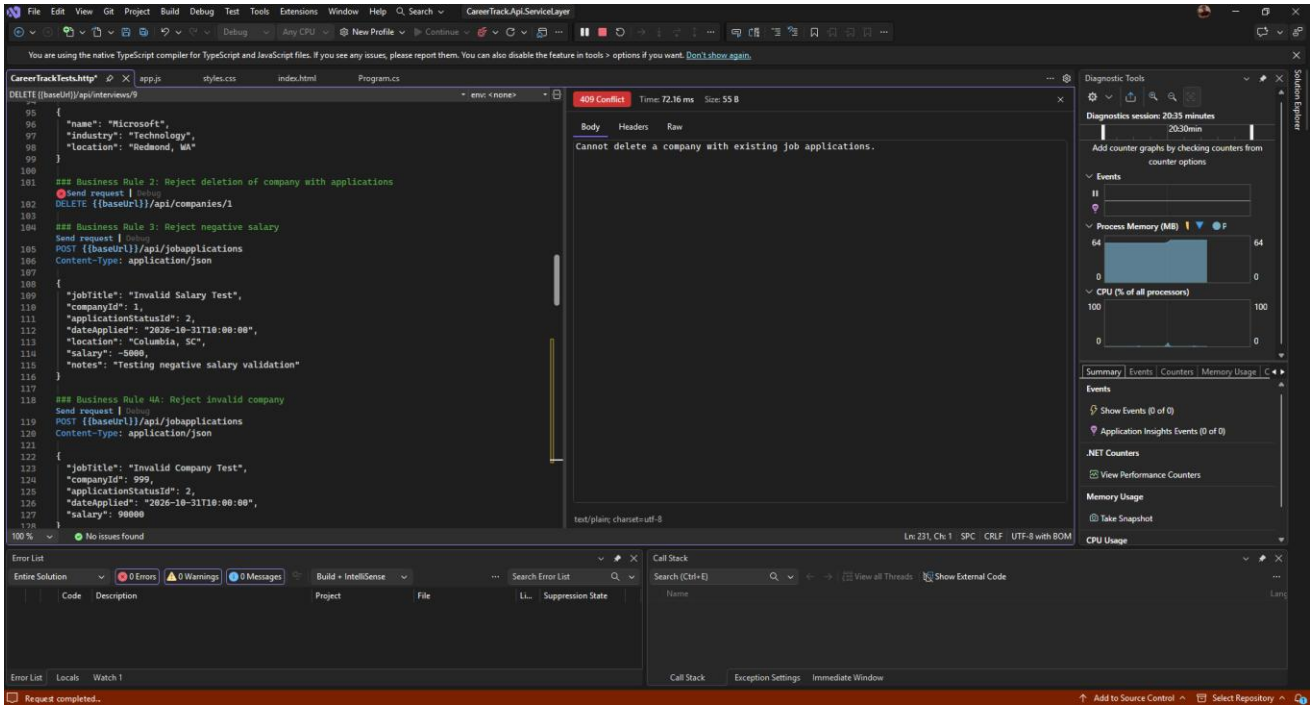


Figure 19. Rule 2 - Reject Deletion of Company with Applications

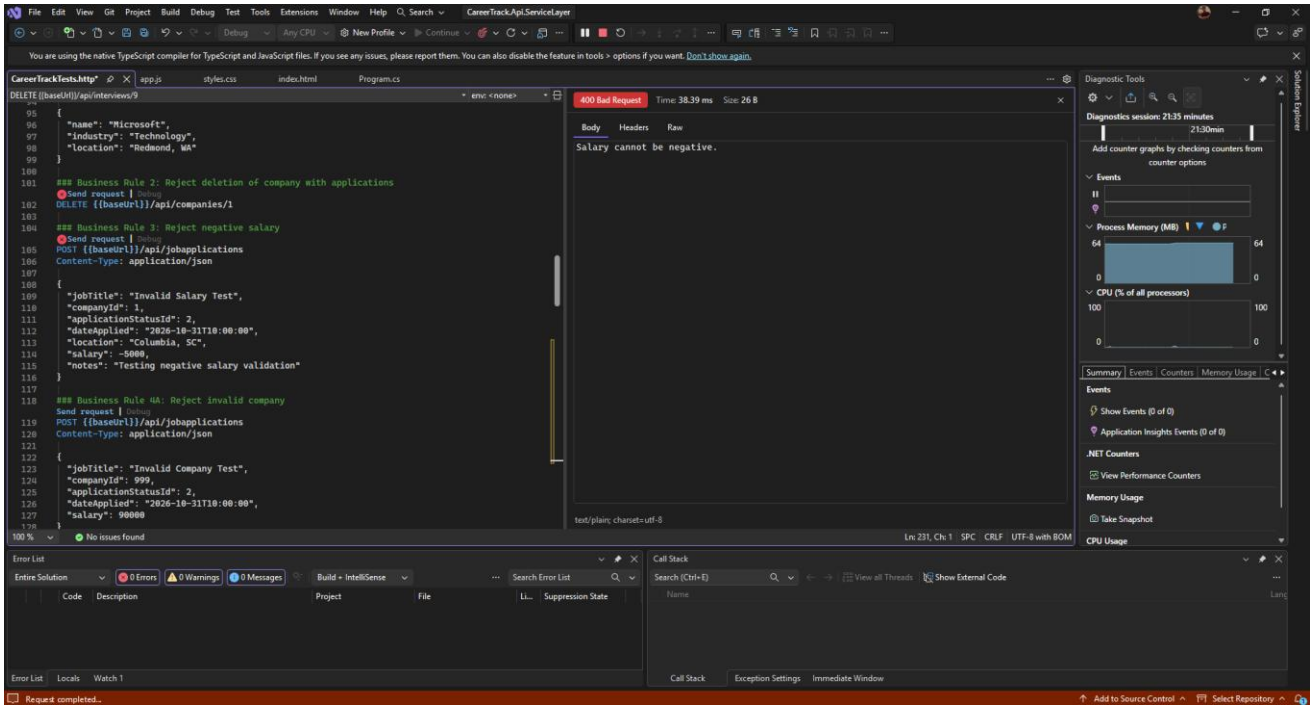


Figure 20. Rule 3 - Reject Negative Salary

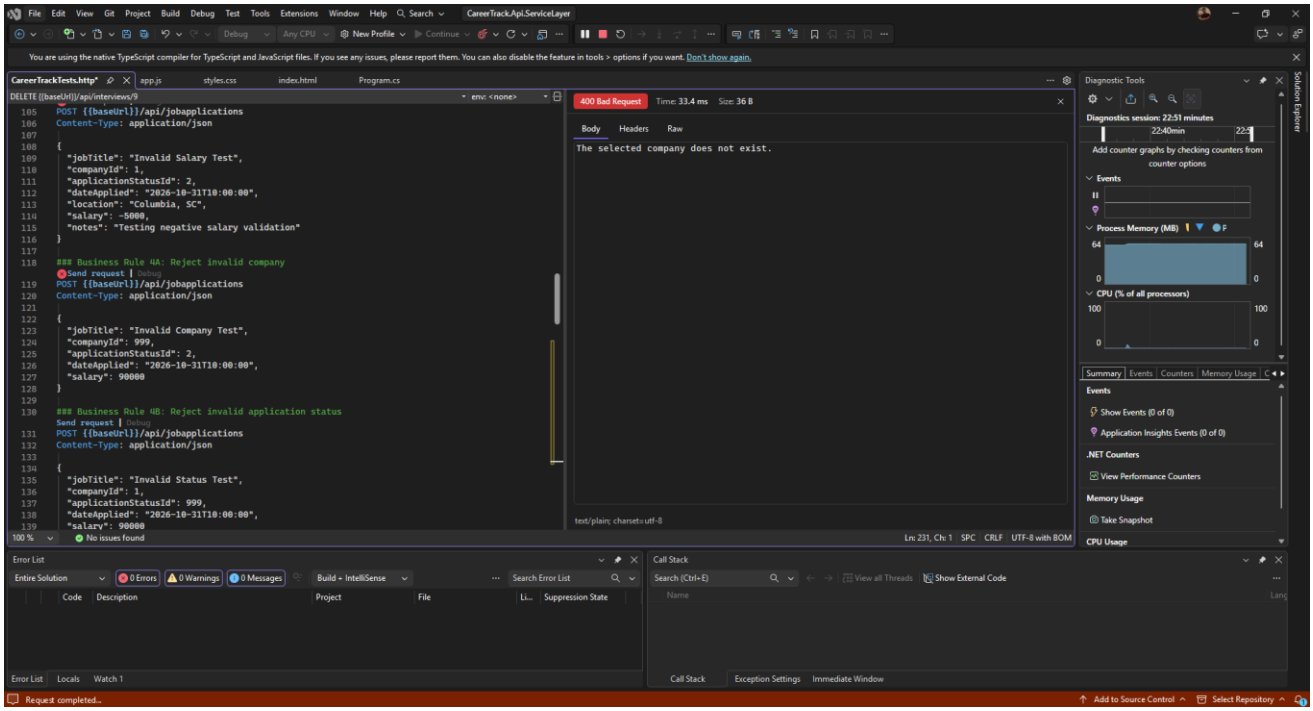


Figure 21. Rule 4A - Reject Invalid Company

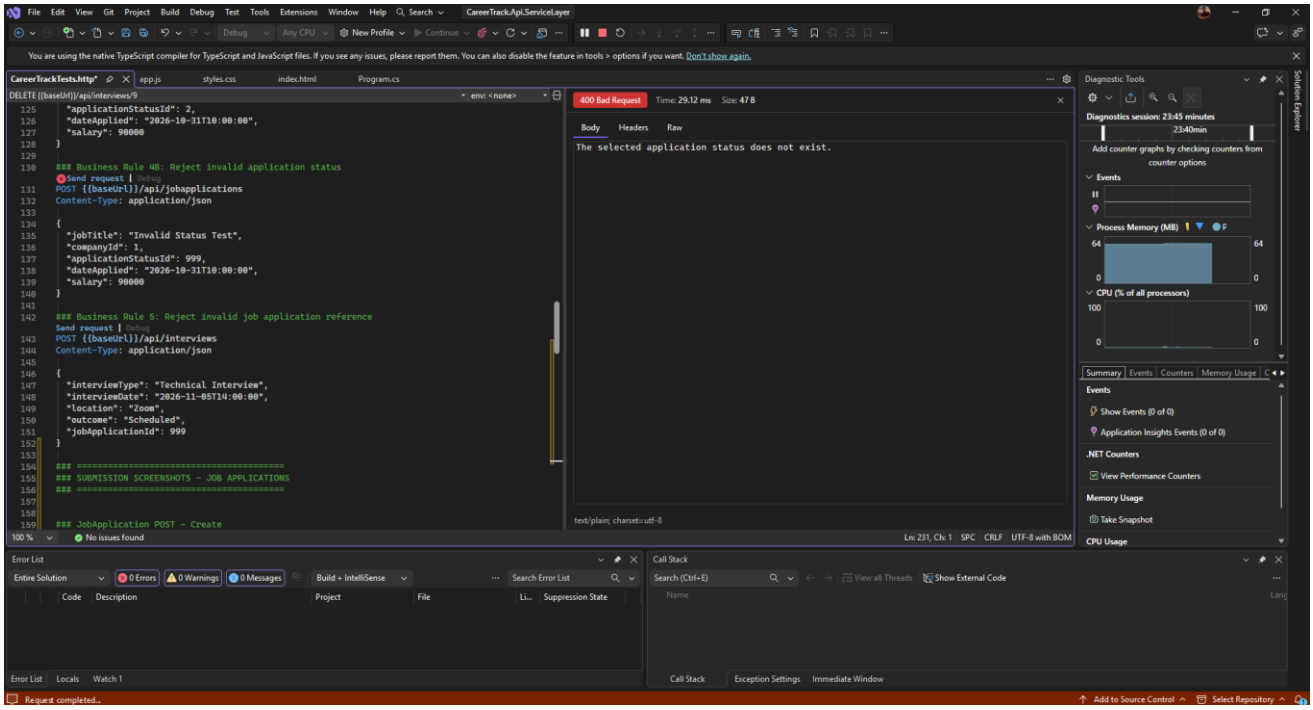


Figure 22. Rule 4B - Reject Invalid Application Status

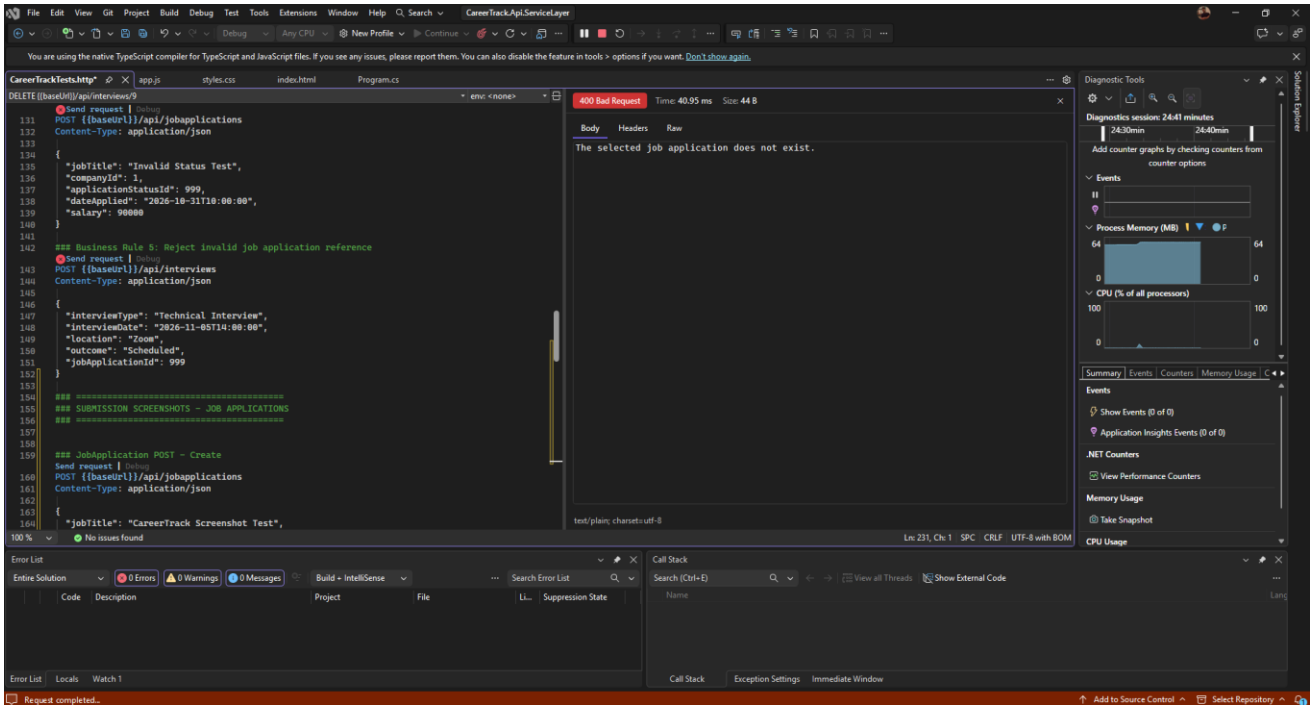


Figure 23. Rule 5 - Reject Invalid Job Application Reference